📦 Creating Classes in C++

> **Class** – a user-defined type that bundles data and functions together.
> **Object** – a concrete instance of a class, occupying memory.

- Use the keyword class to define a new type.
- The class declaration creates a logical abstraction; objects are the physical realizations.

🔐 Access Specifiers

> **Access specifiers** control visibility of class members.

- **public** – members are accessible from any part of the program (the interface).
- **private** – members are accessible only within the class itself; default for classes.
- **protected** – members are accessible within the class and its derived classes; used for inheritance.
- Once a specifier is introduced it remains in effect until another specifier appears or the class ends.
- Specifiers may be repeated multiple times inside a single class declaration.

👩‍💻 Member Functions & Variables

> **Member function** – a function declared inside a class; can access all members, even private ones.
> **Member variable (data member)** – a variable that belongs to a class; also called an *instance variable*.

Restrictions on Class Members

- Non-static members cannot have in-class initializers.
- A class cannot contain an object of its own type (but may contain a pointer to it).
- Members cannot be declared auto, extern, or register.

Accessing Public Variables (Example)

```
#include <iostream>
using namespace std;
```

```
class MyClass {
public:
    int i, j, k;   // public members
};

int main() {
    MyClass a, b;
    a.i = 100;
    a.j = 4;
    a.k = a.i * a.j;
    b.k = 12;
    cout << a.k << " " << b.k;   // prints: 400 12
    return 0;
}
```

🏗️ Struct vs. Class

> **struct** – syntactically similar to a class; the only difference is default member access.

Feature	Class	Struct
Default access spec.	**private**	**public**
Inheritance support	Yes	Yes
Can contain functions	Yes	Yes
Compatibility with C	–	Yes (no member functions)

- Both can hold variables and functions.
- struct exists mainly for C compatibility and for lightweight data aggregates.

🧩 Unions and Anonymous Unions

> **union** – a class-like type where all members share the same memory location.

- Can contain member functions, constructors, and destructors.
- Default access is **public** (compatible with C).

Restrictions on Unions

- No inheritance (cannot be a base or derived class).
- No virtual functions.
- No static data members, reference members, or members with explicit constructors/destructors.

Anonymous Unions

- Have no name; members are accessed directly without a dot.
- Only data members are allowed; cannot contain private/protected members or functions.
- If defined at namespace scope, they must be static.

📊 Comparing Class, Struct, and Union

Feature	Class	Struct	Union
Default access	private	public	public
Inheritance	Yes	Yes	No
Virtual functions	Allowed	Allowed	Not allowed
Static members	Allowed	Allowed	Not allowed
Memory layout	Each member has its own storage	Each member has its own storage	All members share the same storage
Typical use case	Full OOP abstraction	Lightweight data groups (C compatibility)	Memory-efficient storage when only one member is used at a time

🛠️ Object Creation & Instantiation

> **Instantiation** – allocating memory for an object and associating it with its class code.

```
Circle c;        // creates a regular object
Circle *p = new Circle(); // creates a pointer to a dynamically allocated object
```

- Multiple objects can be created from the same class type.

🏗️ Constructors

> **Constructor** – a special public member function invoked automatically when an object is created.

- Same name as the class, no return type.
- Can be overloaded (multiple signatures).

Defining Member Functions

1. **Outside the class** – use the scope resolution operator ::.

```
ReturnType ClassName::FunctionName() { /* ... */ }
```

2. **Inside the class** – written directly; implicitly inline.

```
class Circle {
public:
    void set(double r) { radius = r; }   // automatic inline
};
```

Example (Circle)

```
#include <iostream>
using namespace std;

class Circle {
private:
    double r;
public:
    Circle() { r = 0.0; }
    Circle(int val);
    void set(double val) { r = val; }
    double diameter() { return r * 2; }
    double area();
    double circumference();
};

Circle::Circle(int val) { r = val; }
double Circle::area() { return r * r * (22.0 / 7); }
double Circle::circumference() { return 2 * r * (22.0 / 7); }
```

🤝 Friend Functions & Friend Classes

Friend Function

> A **friend function** is a non-member that can access a class's private and protected members.

```
class Sample {
    int num;
public:
```

```
    void set(int n) { num = n; }
    friend int getDouble(Sample x);   // declaration
};

int getDouble(Sample x) { return x.num * 2; }   // definition
```

- Declared with friend inside the class.
- Not inherited, cannot be static or extern.

Friend Class

> A **friend class** grants all its member functions access to another class's private members.

```
class A {
    int x;
public:
    A(int v) : x(v) {}
    friend class B;   // B can access A::x
};

class B {
public:
    void show(A obj) { cout << "Value of x: " << obj.x << endl; }
};
```

- Friendship is one-way; it does **not** imply inheritance.

⚡ Inline Functions

> **inline** – suggests to the compiler to replace a call with the function's body, eliminating call overhead.

```
inline int max(int a, int b) { return a > b ? a : b; }
```

- Suitable for small, frequently used functions.
- May increase executable size; compiler can ignore the hint for large or complex functions.

Inline Member Functions

- Functions defined inside the class are automatically inline.
- Constructors and destructors can also be inlined.

```
class MyClass {
    int a, b;
public:
    void init(int i, int j) { a = i; b = j; }   // inline
    void show() { cout << a << " " << b << "\n"; } // inline
};
```

📐 Parameterized Constructors

> A **parameterized constructor** accepts arguments to initialize members at creation time.

```
class MyClass {
    int a, b;
public:
    MyClass(int i, int j) { a = i; b = j; }
    void show() { cout << a << " " << b; }
};

int main() {
    MyClass obj(3, 5);
    obj.show();   // prints: 3 5
}
```

📊 Static Data Members

> **static data member** – a variable shared by all objects of a class; only one copy exists.

```
class Test {
    static int sharedValue;   // declaration
    int ownValue;
public:
    void set(int s, int o) { sharedValue = s; ownValue = o; }
    void show();
};

int Test::sharedValue;   // definition
```

- Accessed via ClassName::member or through any object.

Use Cases

- Shared configuration or counters.

- Tracking the number of live objects.

🛠️ Static Member Functions

> **static member function** belongs to the class itself, not to any object.

- Can only access static members; no this pointer.
- Declared with static and called as ClassName::func().

```
class ResourceManager {
    static int inUse;
public:
    static int request() {
        if (inUse) return 0;
        inUse = 1;
        return 1;
    }
    void release() { inUse = 0; }
};

int ResourceManager::inUse;
```

⏳ Constructor & Destructor Timing

- **Constructor** runs when an object is created (global before main, local at declaration).
- **Destructor** runs when an object goes out of scope (reverse order of construction).

```
class Demo {
public:
    Demo(int n) { cout << "Initializing " << n << endl; }
    ~Demo() { cout << "Destructing " << n << endl; }
    int id;
};

Demo g1(1), g2(2);   // global objects

int main() {
    Demo d1(3);
    Demo d2(4);
}
```

Output order demonstrates global → local construction and reverse destruction.

:: Scope Resolution Operator

- Links a class name with a member defined outside the class: ClassName::member.
- Accesses a global name hidden by a local one: ::globalName.

```
int i = 10;          // global
void f() {
    int i = 5;       // local
    ::i = 20;        // modifies global i
}
```

🏛️ Nested & Local Classes

Nested Class

- Defined inside another class; visible only within the enclosing class's scope.
- Rarely needed; inheritance often provides a cleaner solution.

Local Class

- Defined inside a function; visible only within that function.
- Restrictions: all members must be defined inside the class; cannot access non-static locals of the enclosing function.

```
void func() {
    class Local {
        int i;
    public:
        void set(int v) { i = v; }
        int get() const { return i; }
    } obj;
    obj.set(10);
    cout << obj.get();   // prints 10
}
```

📤 Passing & Returning Objects

Passing by Value

- Copies the object using the copy constructor; destructor runs for the copy when the function returns.

```
void foo(MyClass obj);   // copy made
```

Returning an Object

- Creates a temporary to hold the return value; the temporary is destroyed after the full expression.

```
MyClass make() {
    MyClass temp;
    return temp;   // copy/move may occur
}
```

🔁 Object Assignment

- The = operator performs a shallow copy of data members (default behavior) unless a custom assignment operator is defined.

```
MyClass a, b;
a.set_i(99);
b = a;   // copies i from a to b
```

🖥️ Interface Definition

> **Interface** – the public part of a class (public functions and variables) that other code interacts with.

```
class Example {
private:
    int data;
public:
    void setData(int d) { data = d; }   // public interface
    int getData() const { return data; }
};
```

- Encapsulation hides implementation details while exposing a controlled set of operations.